Handoff: ovt_places_usa IP/RDAP enrichment

Oct 23, 2026 · @Boris

Project status

We are building an accurate geotargeting system.

A. Targeting by DMA works.
B. [Priority 1] For ZIP-code-to-IP-address translation, we are using landmarks (businesses, local government offices, schools, universities, libraries that host their webservers on-premises), currently from Overture (stored in table ovt_places_usa) but soon also from other sources, mainly Foursquare. However, static on-premises IP addresses, which must stay with the customer, are assigned through different ranges from dynamically issued (DHCP) IP addresses, which are often automatically moved to other geographic locations in the carrier's footprint. But for each geographic location, the static and dynamic IP addresses are routed together. I propose the following algorithm (critique if necessary):
    1. For each ovt_places_usa.onpremises=true address, use mtr to find the carrier node immediately adjacent to customer premises equipment (CPE). The penultimate node usually, but not always, has a domain name in one of the carrier's domains; when it does not, the CPE often has a domain name in the customer's domain; when neither the last carrier node not the CPE have domain names, it is usually safe to assume that the penultimate node is the last carrier node before CPE.
    2. For each ip-api.com–probed address, obtain the carrier node immediately adjacent to CPE.
    3. If that node matches one in the landmarks table ovt_places_usa, assign the Overture ZIP code of the landmark to the range in table ip2city_dbiplite_probe_tbl . Note that this can be done independently of probing if the randomly selected address is selected in advance of probing, which it should be if it isn't already. Note also that one node can route to multiple ZIP codes; this is common in rural areas. Duplicate the record in ip2city_dbiplite_probe_tbl and assign the additional ZIP code to the duplicate if it is found.
    4. Count the ip2city_dbiplite_probe_tbl ranges for which we have ZIP codes and those for which we don't.
    5. Count the ZIP codes for which we have ranges and those for which we don't (a complete list of ZIP codes is in table city2zip_tbl).
C. [Priority 2] I expect that many rural ZIP codes will have no on-premises landmarks in Overture data. We will need to load and process the following additional sources (for simplicity, we can use the ovt_places_usa table, and change its name later):
    1. Foursquare
    2. OpenStreetMap (via Geofabrik extracts or Overpass)
    3. Wikidata
    4. National Park Service API

5. National Register of Historic Places
6. IMLS Public Libraries Survey
7. NCES school data
8. NRHP

D. [Priority 3] API to translate DMA numbers to lists of IP addresses is supposedly ready from earlier in this session but hasn't been deployed. It should be deployed.
E. [Priority 3] GUI to translate DMA numbers to lists of IP addresses (with DMAs selectable from a nonexclusive list by name) is supposedly ready from earlier in this session but hasn't been deployed. It should be deployed.
F. [Priority 4] All of the landmark sources should be reloaded and reprocessed periodically. I understand that Overture is updated daily. Determine the update frequency of Overture and the other sources, and write the reloading and reprocessing code.
G. For now, maintain the probing with ip-api.com . Its results are several miles distant from the actual locations of ranges, but may be a necessary fallback if we cannot get to nationwide landmark coverage.

Summary

Hostname-to-IPv4 resolution of Overture data is finished: 6,655,429 of 7,911,403 distinct hosts resolved, and 12,334,433 rows of `ovt_places_usa` now carry an `ip_address`. The RDAP registrant lookup (543,336 distinct IPs) has not yet run at scale: `rdap-lookup.sh` is fixed and gives identical output on Linux Mint and macOS for the one IP compared, and a 30-IP random sample is the next step. The `onpremises` flag is not set yet; it waits for the provider list Boris chooses from the `url_location` report.

Scope is US data: IPs outside ARIN's range (HTTP 303 from ARIN) are recorded as failed lookups and left for later.

Files and locations

Everything runs on the Linux Mint 22.3 workstation (Immetrica-Mint30); the macOS machine (Immetrica-OSX40) was used only to compare `rdap-lookup.sh` output.

File or table	Where	Purpose
`rdap-lookup.sh`	`~/geo/rdap-lookup/cmd/` on both machines	ARIN RDAP lookup for one IP. Checksum `3459968002 4609` on both.
`ovt_resolve_hosts.py`	Linux, in the directory Boris confirmed	`resolve`, `rdap`, `report`, `mark` and `all` subcommands. Standard library only; all database access goes through `psql`.
`parquet_to_pg.py`	`~/geo/overture_fetch/`	Parquet-to-PostgreSQL loader; its generated `CREATE TABLE`

		now includes the six new columns.
`rdap-lookup-follow.sh`	`~/geo/rdap-lookup/cmd/`	Redirect-following copy; out of scope for now.
`ovt_places_usa`	PostgreSQL	Main table: Overture places plus the six new columns.
`ovt_places_usa_host_ip`, `ovt_places_usa_ip_rdap`	PostgreSQL	Caches created by the script: host to IP, and IP to RDAP result or error. Make the steps resumable.

Procedure

Steps 1 and 2 are done; the remaining work is steps 3 to 8, in order. Prerequisites: `psql` already connected to the right database, `jq` and `curl` on every machine that runs `rdap-lookup.sh`, and `pip install duckdb` only if `parquet_to_pg.py` is used.

1. Resolve hostnames to IPv4 (done)

```
python3 ovt_resolve_hosts.py resolve
```

It reads the first website of each place, looks up each distinct host once, and updates `ip_address`. The last run covered 7,911,403 hosts in about 22,200 seconds. It is resumable: Ctrl-C saves finished lookups, a rerun handles only new hosts, and `--retry-failed` retries earlier failures.

2. Reload the Overture data

2.a. Complete reload:

```
psql -X -c "TRUNCATE ovt_places_usa_ip_rdap" -c "UPDATE ovt_places_usa SET
url_location = NULL WHERE url_location IS NOT NULL"
python3 ovt_resolve_hosts.py rdap --help | grep -- --random
```

The second command confirms you are in the directory holding the current `ovt_resolve_hosts.py`: it must print a line containing `--random`.

2.b. Delta (needed for production; not yet implemented)

The Overture data contains additions, deletions and changes at least daily. Write procedure to identify them relative to the current contents of `ovt_places_usa` and upsert/delete accordingly. Current contents of table should be archived for auditability in a clone of the table (the clone should include a timestamp with the archival date.

3. Run the full RDAP lookup

```
python3 ovt_resolve_hosts.py rdap
```

It covers 543,336 distinct IPs with 4 threads by default and is resumable. Expect about 20 hours (an estimate from one 0.54-second call, before any rate limiting). The default script path is `~/geo/rdap-lookup/cmd/rdap-lookup.sh`. `python3 ovt_resolve_hosts.py all` chains steps 2, 6 and 7.

4. Produce the provider report

```
python3 ovt_resolve_hosts.py report
```

Prints every distinct `url_location` value with its row count and writes `ovt_places_usa_url_location_counts.csv`.

5. Flag providers as off-premises

Create a table (for which issue SQL code) and populate it with the records in the repo `geo/overture_fetch/isp_categorization/definitive_isps.txt` .

The current `ovt_resolve_hosts.py` should set `onpremises` as follows:

- If `status <> true`, leave NULL
- Else (if `status = true`):
- Set `onpremises = true` if: {
    - registrant is an ISP (residential or business) AND there is no second registrant;
    - there is a match on one of the two RDAP ZIP codes;
    - there are two registrants and any non-ISP registrant's RDAP ZIP code is adjacent to the landmark's Overture ZIP code (zip_adjacency table contains adjacent pairs in columns ZIP_A, ZIP_B) }
- Set `onpremises = false` in all other cases (including where there are two registrants, one of which is an ISP, and the other's ZIP code does not match or is adjacent to the landmark's Overture ZIP code).

Progress check at any point

```
psql -X -c "SELECT count(*) FILTER (WHERE ip_address IS NOT NULL) AS with_ip,
count(*) FILTER (WHERE url_location IS NOT NULL) AS with_location, count(*)
FILTER (WHERE onpremises = false) AS off_premises FROM ovt_places_usa"
```

Rebuilding the table from parquet (only if needed)

```
python3 parquet_to_pg.py schema --input
/home/boris/geo/overture_fetch/out/usa_places.parquet --table ovt_places_usa
psql -v ON_ERROR_STOP=1 -f ovt_places_usa_schema.sql
python3 parquet_to_pg.py load --input
/home/boris/geo/overture_fetch/out/usa_places.parquet --table ovt_places_usa
psql -v ON_ERROR_STOP=1 -f ovt_places_usa_postload.sql
```

Open items and decisions

Out of scope for now: following redirects to other registries, and IPv6 (resolution is IPv4 only). Nothing yet populates `ip_last_carrier_node`, `ip_address_range` or `ip_last_carrier_node_range`; they exist only as columns.